

GEBZE TECHNICAL UNIVERSITY

Department of Computer Engineering

CSE 344 - System Programming

Spring 2026

Magic Academy Server & Clients

Homework #6 Report

Student Name: Ayşe Feyza SERBEST

Student ID: 220104004052

Course: CSE 344

Date: May 20, 2026

Contents

1	System Design	2
1.1	Single-Threaded Multi-Client Management Loop	2
1.2	Recalculation of Idle Timeout Before Every Iteration	2
1.3	Capacity Enforcement and Immediate Rejection	3
2	Data Structures	4
2.1	The <code>client_t</code> Struct Definition	4
2.2	Global Ingredient Envanter and Personal Spellbooks Management	4
2.3	Inventory Update Logic and Consumption Constraints	4
3	Partial Read/Write Handling	6
3.1	Line Buffer Accumulation and Token Shifting Architecture	6
3.2	Partial Write Management	6
4	Signal Handling and Shutdown	7
4.1	Server Shutdown Cascade (Ctrl+C Trigger)	7
4.2	Client Exit Sequence (Ctrl+C Trigger)	7
5	Test Scenarios	8
5.1	Scenario 1: Error Processing, Authorization, and Premature Messaging	8
5.2	Scenario 2: Dynamic Brewing, Personal-vs-Global Stocks Isolation	10
5.3	Scenario 3: Maximum Boundary Satiation and Rejection Safeguards	13
5.4	Scenario 4: Timeout Purging Mechanisms and Minimum Delay Selection	16
6	Challenges and Engineering Solutions	21
6.1	Challenge 1: POSIX Structure Masking Under Strict C11 Standards	21
6.2	Challenge 2: Multi-Command Packet Aggregation and Stream Boundary Collisions	21

1 System Design

1.1 Single-Threaded Multi-Client Management Loop

The `hogwarts` server is designed to handle multiple simultaneous TCP client connections within a single execution thread, strictly avoiding the *thread-per-client* architecture. To achieve high-concurrency network I/O multiplexing, the server employs the Linux `select()` system call.

The architecture centers around an infinite loop that constantly updates and monitors a set of file descriptors (`fd_set`). The loop proceeds through the following structured sequence:

1. **FD Set Initialisation:** The read file descriptor set (`rfds`) is cleared using `FD_ZERO(&rfds)`. The primary listening socket (`listen_fd`) is added to the set via `FD_SET(listen_fd, &rfds)`.
2. **Active Clients Extraction:** The server iterates through its active client management array (`g_clients`). Every valid file descriptor ($fd \geq 0$) is registered into the `rfds` tracking set. Simultaneously, the maximum file descriptor number (`maxfd`) is tracked to provide an accurate bounding constraint for the subsequent `select()` call.
3. **Kernel-Level Multiplexing Blocks:** The server issues the `select(maxfd + 1, &rfds, NULL, NULL, &tv)` call. This suspends execution until either data arrives on at least one registered socket or the calculated idle timeout expires.
4. **Asynchronous Event Dispatching:** Upon return, the kernel modifies `rfds` to indicate active channels. If `listen_fd` is flagged, the server branches to accept a new client connection. If any client socket is flagged, the server enters the non-blocking parsing engine to ingest network streams.

1.2 Recalculation of Idle Timeout Before Every Iteration

To satisfy the constraint of automatically disconnecting clients that remain inactive for longer than `TIMEOUT` seconds without introducing separate timer threads or blockages like `sleep()`, the server performs an explicit mathematical remaining-time reduction before every single invocation of `select()`.

The timeout calculation algorithm follows these steps:

$$t_{\text{current}} = \text{time}(\text{NULL}) \quad (1)$$

For each active client $i \in [0, \text{max_clients})$, the elapsed inactive duration is evaluated:

$$\Delta t_i = t_{\text{current}} - t_{\text{last_active},i} \quad (2)$$

If $\Delta t_i \geq T_{\text{timeout}}$, the client is instantly disconnected. The server sends a `TIMEOUT DISCONNECT\n` line, closes the socket descriptor, logs the transaction, and marks the slot as free. Otherwise, the remaining safe window for this client is:

$$t_{\text{remain},i} = T_{\text{timeout}} - \Delta t_i \quad (3)$$

The server tracks the absolute minimum window across all connected nodes:

$$t_{\text{select_timeout}} = \min_i(t_{\text{remain},i}) \quad (4)$$

This calculated minimum integer boundary is assigned to the `struct timeval tv` parameter (`tv.tv_sec`). Consequently, the kernel guarantees to wake the process exactly when the earliest client is on the verge of breach, preventing any cpu-bound spin-waiting.

1.3 Capacity Enforcement and Immediate Rejection

The server strictly enforces the maximum client capacity configured via the `-n <max_clients>` CLI flag. When a new connection arrives at the listening socket while the active slot registry is fully occupied, the server mitigates structural blocking by applying an immediate rejection paradigm:

- The incoming connection is extracted from the TCP backlog queue using `accept()`. This releases the operating system's kernel buffer and prevents socket resource starvation.
- The server identifies that no empty slot is available in `g_clients`.
- An error packet containing `ERR HOGWARTS_FULL\n` is pushed out to the client socket immediately via a robust, blocking `write_all()` sweep.
- The rejection is logged immediately to the standard output and the file logger matching the mandatory schema: `[SERVER] REJECTED fd=N ip=X.X.X.X reason=HOGWARTS_FULL`.
- The connection is terminated cleanly using `close()` without dedicating any heap storage or memory allocations to the unauthorized entity.

2 Data Structures

2.1 The `client_t` Struct Definition

The fundamental state record allocated for every simultaneous TCP client connection is encapsulated within the `client_t` structure. This tracks partial incoming lines, role validation flags, and distinct inventory states.

```

1 typedef struct {
2     int          fd;                      /* File descriptor; -1
   implies empty slot */
3     char         username[32];           /* Unique client name from
   ENROLL */
4     char         type[16];               /* Client role: "WIZARD"
   or "PROFESSOR" */
5     int          enrolled;              /* Enrollment boolean
   status flag */
6     char         line_buf[512];         /* Stream read line buffer
   for partial inputs */
7     size_t       line_len;              /* Current length of valid
   bytes inside line_buf */
8     time_t       last_active;           /* Timestamp recording the
   last valid command */
9     char         ip_str[16];            /* Human-readable remote
   IP string for logging */
10    spell_entry_t spellbook[64];         /* Personal inventory
   table (WIZARD only) */
11    int          spell_count;            /* Count of active
   ingredients inside spellbook */
12 } client_t;

```

Listing 1: Structure definition of `client_t` state management record

2.2 Global Ingredient Envanter and Personal Spellbooks Management

The server isolates state representation into two separate, completely orthogonal storage types:

1. **Global Ingredient Inventory Table:** Represented by a global array `ingredient_t g.ingredients[256]`, populated at startup by parsing `ingredients.txt`. This table represents the school's overall warehouse capacity. It changes dynamically when orders are validated. Professors fetch representations of this table using the `INSPECT` and `SCROLL` commands.
2. **Per-Wizard Spellbooks:** Embedded within the `client_t` record as an array of `spell_entry_t`. This maps a wizard's unique personal inventory balances.

2.3 Inventory Update Logic and Consumption Constraints

The interaction rules ensure that personal accounts govern transaction approvals:

- **BREW <ingredient> <qty>:** Increases the shared warehouse pool counter (`g.ingredients[idx].qty`) and simultaneously expands the wizard's individual `spellbook` record for that element. Unknown ingredients outside the initial global initialization set trigger an immediate `ERR UNKNOWN_INGREDIENT` error.
- **CONSUME <ingredient> <qty>:** This action invokes a strict personal constraint check. The server searches the wizard's `**personal spellbook**` to verify possession. If the wizard's local balance is less than the requested `qty`, the transaction is rejected instantly with `ERR INSUFFICIENT_INGREDIENTS`—even if the global warehouse has ample stocks. If validated,

the quantity is decremented symmetrically from both the global table and the local wizard spellbook.

3 Partial Read/Write Handling

Because TCP is a stream-oriented protocol that delivers arbitrary byte slices without observing application message boundaries, messages can arrive fragmented across network packets or concatenated into a single buffer read.

3.1 Line Buffer Accumulation and Token Shifting Architecture

To handle these network anomalies safely without dropped bytes or stalled inputs, the server integrates a linear line-buffer tracking mechanism via `client_t.line_buf` and `client_t.line_len`:

1. When a read event triggers, the server computes the current available space: `space = 512 - line_len`. If no space remains, an error (`ERR_TOOLONG`) is transmitted, and the client is closed.
2. The server reads up to `space` bytes into `line_buf + line_len`. The read length is appended to `line_len`.
3. The buffer is scanned using `memchr()` for the newline delimiter (`\n`).
4. If no newline is found, the server terminates the branch and gracefully returns to the main `select()` framework to wait for more packets.
5. When a newline character is located at offset L , a full string extraction occurs. The content is sliced into a null-terminated string, stripping accompanying carriage returns (`\r`).
6. The command token parser processes the command via `handle_command()`.
7. Crucially, to handle multi-command blocks or leftovers, the unconsumed remainder of the buffer is shifted forward using memory block relocation:

```

1  size_t consumed = L + 1;
2  size_t remain = line_len - consumed;
3  if (remain > 0) {
4      memmove(line_buf, line_buf + consumed, remain);
5  }
6  line_len = remain;
7

```

This loop continues processing sequentially as long as newlines are discovered.

3.2 Partial Write Management

Symmetrically, socket output channels can experience congestion, causing standard `write()` calls to transmit fewer bytes than requested. To prevent transmission losses, the application provides a non-interruptible network writer loop, `write_all()`:

```

1  ssize_t write_all(int fd, const char *buf, size_t len) {
2      size_t total = 0;
3      while (total < len) {
4          ssize_t n = write(fd, buf + total, len - total);
5          if (n < 0) {
6              if (errno == EINTR) continue;
7              return -1;
8          }
9          if (n == 0) return -1;
10         total += (size_t)n;
11     }
12     return (ssize_t)total;
13 }

```

4 Signal Handling and Shutdown

The system features robust signal management to ensure clean resource reclamation, socket unbinding, and immediate shutdown cascades upon receiving terminal termination events.

4.1 Server Shutdown Cascade (Ctrl+C Trigger)

When an operator issues a termination signal (SIGINT) to the parent `hogwarts` process, the state sequence follows a structured shutdown pattern:

1. The system triggers the handler mapped via a clean `sigaction` declaration. To respect async-signal-safety, the handler avoids rich output formatting functions like `printf()` and simply sets a global flag:

```
1 static void on_sigint(int signo) { g_shutdown_flag = 1; }
2
```

2. The kernel interrupts the active `select()` block, forcing it to immediately return `-1` with `errno` set to `EINTR`.
3. The main loop notices that `g_shutdown_flag` evaluates to `1` and breaks from its operational loop.
4. The server iterates through all active file descriptors in the client array, sends a termination notification message (`SERVER SHUTDOWN\n`), and invokes `close()` to free the descriptors.
5. The server outputs the formal lifecycle records into the logging framework:
 - `[SERVER] SHUTDOWN signal=SIGINT`
 - `[SERVER] CLEANUP_DONE clients=0`
6. The listening master socket is closed, unbinding the TCP port cleanly, and the process terminates with status `0`.

4.2 Client Exit Sequence (Ctrl+C Trigger)

When a user targets a running `wizard` or `professor` runtime environment with a SIGINT event:

1. The local client handler sets an internal shutdown flag, breaking the user-space I/O loop.
2. The client process writes an explicit protocol message over the socket: `APPARATE\n`.
3. The server reads the token, executes an immediate state purge for that client slot, records a formal log message (`reason=APPARATE`), and replies with a confirmation string: `OK APPARATE\n`. The client reads this response line, prints the termination receipt, closes the socket descriptor, and terminates.

5 Test Scenarios

5.1 Scenario 1: Error Processing, Authorization, and Premature Messaging

This scenario tests authentication constraints, role-based boundary rejections, and attempts to bypass enrollment checks by submitting premature commands via raw network connections (nc).

1. Hogwarts Server Terminal:

The single-threaded server processing network multiplexing and authorization logs.

```
→ HW6 git:(main) X ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-20 20:55:13] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s
[2026-05-20 20:55:16] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 20:55:16] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 20:55:19] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 20:55:19] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 20:55:22] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 20:55:29] [SERVER] CLIENT_DISCONNECTED username=- reason=APPARATE
[2026-05-20 20:55:31] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 20:55:37] [SERVER] CLIENT_DISCONNECTED username=- reason=hangup
[2026-05-20 20:55:59] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 20:56:00] [SERVER] CLIENT_DISCONNECTED username=hermione reason=APPARATE
```

Figure 1: Scenario 1 - Server Terminal Output

2. Hermione (Wizard) Terminal:

Showing illegal command attempts and unauthorized boundary rejections.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
FLY BROOMSTICK
[WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY
INSPECT MOONSTONE
[WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
SCROLL
[WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
^C[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X
```

Figure 2: Scenario 1 - Hermione Client Terminal

3. Dumbledore (Professor) Terminal:

Showing unauthorized attempt to execute BREW and CONSUME functions.

```
→ HW6 git:(main) X ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
BREW MOONSTONE 5
[PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
CONSUME MOONSTONE 5
[PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X
```

Figure 3: Scenario 1 - Dumbledore Client Terminal

4. Netcat (Raw Connection) & Duplicate Login Terminal:

Demonstrating name_taken rejection and raw TCP data constraint enforcement.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED ERR ENROLL name_taken
^C[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X nc 127.0.0.1 5000
FLY BROOMSTICK
ERR NOT_ENROLLED
^C
→ HW6 git:(main) X
```

Figure 4: Scenario 1 - Raw TCP and Duplicate Registration Rejection

Observed Output Behaviors:

- Connecting a second client with an identical name (**hermione**) successfully prompts an immediate rejection with **ERR ENROLL name_taken**.
- Emitting arbitrary data over raw **nc** before registering returns the proper constraint violation: **ERR NOT_ENROLLED**.
- Professors trying to execute a **BREW** statement receive an explicit **ERR UNAUTHORIZED** code block.

5.2 Scenario 2: Dynamic Brewing, Personal-vs-Global Stocks Isolation

This scenario validates correct execution of production flows, personal inventory increments, warehouse reporting, and individual deficit boundaries.

1. Hogwarts Server Terminal:

The central ledger dynamically processing stock updates and logging transactions.

```

→ HW6 git:(main) X ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
[2026-05-20 21:23:54] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s
[2026-05-20 21:24:00] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:24:00] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 21:24:06] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 21:24:06] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-20 21:24:16] [SERVER] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-20 21:24:30] [SERVER] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-20 21:24:36] [SERVER] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-20 21:24:45] [SERVER] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-20 21:24:51] [SERVER] SCROLL professor=mcgonagall ingredients=5
[2026-05-20 21:25:18] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-20 21:25:28] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[2026-05-20 21:25:29] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE

```

Figure 5: Scenario 2 - Server Transaction Log

2. Harry (Wizard) Terminal:

Validating dynamic inventory (BREW), personal spellbook updates, and deficit controls (CONSUME).

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
BREW MOONSTONE 10
[WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
BREW UNICORN_HAIR 15
[WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
CONSUME MOONSTONE 5
[WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
CONSUME UNICORN_HAIR 20
[WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
BREW BUTTERBEER 5
[WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |
```

Figure 6: Scenario 2 - Harry Client Terminal

3. McGonagall (Professor) Terminal:

Executing INSPECT, SCROLL, and ROSTER query blocks over the global arrays.

```
→ HW6 git:(main) X ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED OK INSPECT DRAGON_SCALE 50
SCROLL
[PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEA
THER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER harry,mcgonagall
INSPECT BUTTERBEER
[PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT
^C[PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |
```

Figure 7: Scenario 2 - McGonagall Client Terminal

5.3 Scenario 3: Maximum Boundary Satiation and Rejection Safeguards

This scenario validates the server's stability when the active connection limit (`-n 3`) is reached, ensuring it rejects additional connections gracefully without dropping existing active connections.

1. Hogwarts Server Terminal:

Showing execution logs during maximum saturation, rejection of excess clients, and slot recovery.

```

→ HW6 git:(main) X ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 60
[2026-05-20 21:51:48] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 3 | Timeout: 60s
[2026-05-20 21:51:55] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:51:55] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 21:52:01] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 21:52:01] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 21:52:06] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 21:52:06] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-20 21:52:11] [SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 21:52:17] [SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 21:52:27] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-20 21:52:34] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:52:34] [SERVER] ENROLL username=neville type=WIZARD fd=5
[2026-05-20 21:52:42] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 21:52:58] [SERVER] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-20 21:52:59] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 21:53:00] [SERVER] CLIENT_DISCONNECTED username=neville reason=APPARATE

```

Figure 8: Scenario 3 - Server Capacity Enforcement Trace

2. Harry (Wizard) Terminal (Slot 1):

Occupying the first server slot and later disconnecting to free up capacity.

```

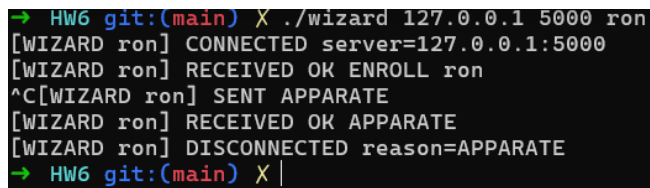
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
^C[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |

```

Figure 9: Scenario 3 - First Active Client Session (Harry)

3. Ron (Wizard) Terminal (Slot 2):

Occupying the second server slot.

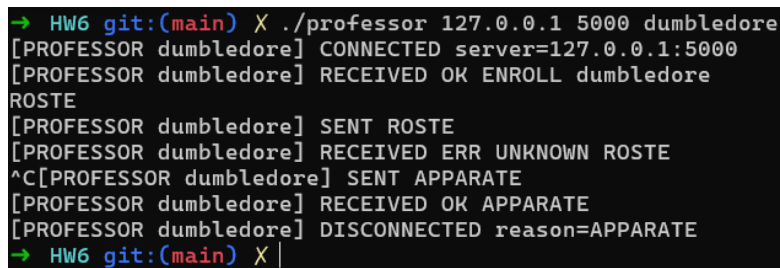
A terminal window showing the execution of the 'wizard' command. The prompt is 'HW6 git:(main) X'. The command './wizard 127.0.0.1 5000 ron' is entered. The output shows a successful connection to the server at 127.0.0.1:5000, followed by 'RECEIVED OK ENROLL ron', 'SENT APPARATE', 'RECEIVED OK APPARATE', and 'DISCONNECTED reason=APPARATE'. The prompt returns to 'HW6 git:(main) X |'.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
^C[WIZARD ron] SENT APPARATE
[WIZARD ron] RECEIVED OK APPARATE
[WIZARD ron] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |
```

Figure 10: Scenario 3 - Second Active Client Session (Ron)

4. Dumbledore (Professor) Terminal (Slot 3):

Occupying the final server slot and executing the ROSTER command.

A terminal window showing the execution of the 'professor' command. The prompt is 'HW6 git:(main) X'. The command './professor 127.0.0.1 5000 dumbledore' is entered. The output shows a successful connection to the server at 127.0.0.1:5000, followed by 'RECEIVED OK ENROLL dumbledore', 'ROSTE', 'SENT ROSTE', 'RECEIVED ERR UNKNOWN ROSTE', 'SENT APPARATE', 'RECEIVED OK APPARATE', and 'DISCONNECTED reason=APPARATE'. The prompt returns to 'HW6 git:(main) X |'.

```
→ HW6 git:(main) X ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
ROSTE
[PROFESSOR dumbledore] SENT ROSTE
[PROFESSOR dumbledore] RECEIVED ERR UNKNOWN ROSTE
^C[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |
```

Figure 11: Scenario 3 - Third Active Client Session and Roster Query (Dumbledore)

5. Overflow & Replacement Terminal:

Demonstrating immediate HOGWARTS_FULL rejections (Voldemort, Snape) and subsequent successful enrollment (Neville) after a slot is freed.

```
→ GTU-CSE344-Systems-Programming git:(main) X cd HW6
→ HW6 git:(main) X ls
CSE344_hw6.pdf Makefile hogwarts ing ingredients.txt ingredients_dirty.txt pr
ofessor server.log wizard
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5000
[WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
→ HW6 git:(main) X ./professor 127.0.0.1 5000 snape
[PROFESSOR snape] CONNECTED server=127.0.0.1:5000
[PROFESSOR snape] RECEIVED ERR HOGWARTS_FULL
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5000
[WIZARD neville] RECEIVED OK ENROLL neville
BREW MOONSTONE 5
[WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
^C[WIZARD neville] SENT APPARATE
[WIZARD neville] RECEIVED OK APPARATEONE 5
[WIZARD neville] DISCONNECTED reason=APPARATE
→ HW6 git:(main) X |
```

Figure 12: Scenario 3 - Client Overload Rejection and Slot Recovery

5.4 Scenario 4: Timeout Purging Mechanisms and Minimum Delay Selection

This scenario tests the asynchronous timeout engine by allowing client nodes to sit completely silent past the configured timeout constraint. It also validates the absence of zombie processes and memory leaks upon server termination.

1. Hogwarts Server Terminal (Live Output & Process Check):

Demonstrating dynamic timeout calculations, client purges, and the final `ps aux` validation proving a completely clean exit (zero zombie processes).

```

➔ HW6 git:(main) X ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
[2026-05-20 22:11:11] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 10s
[2026-05-20 22:11:28] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:11:28] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 22:11:30] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:11:30] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-20 22:11:33] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 22:11:40] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-20 22:11:40] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 22:11:42] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-20 22:11:55] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:11:55] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 22:11:58] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-20 22:11:58] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-20 22:12:05] [SERVER] TIMEOUT username=ron fd=6 elapsed=10s
[2026-05-20 22:12:05] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-20 22:12:15] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:12:15] [SERVER] ENROLL username=ron type=WIZARD fd=5
[2026-05-20 22:12:14] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-20 22:12:19] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:12:19] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 22:12:20] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-20 22:12:24] [SERVER] TIMEOUT username=ron fd=5 elapsed=10s
[2026-05-20 22:12:24] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-20 22:12:30] [SERVER] TIMEOUT username=dumbledore fd=6 elapsed=10s
[2026-05-20 22:12:30] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-20 22:12:31] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:12:31] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=5
[2026-05-20 22:12:33] [SERVER] ROSTER professor=mcgonagall clients=1
[2026-05-20 22:12:43] [SERVER] TIMEOUT username=mcgonagall fd=5 elapsed=10s
[2026-05-20 22:12:43] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
^C[2026-05-20 22:12:52] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 22:12:52] [SERVER] CLEANUP_DONE clients=0
➔ HW6 git:(main) X
➔ HW6 git:(main) X ps aux | grep hogwarts
afeyz 5033 0.0 0.0 4096 1760 pts/6 S+ 22:12 0:00 grep --color=auto --exclude-dir=.bzr --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn --exclude-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=.venv hogwarts

```

Figure 13: Scenario 4 - Server Live Trace and Process Verification

2. Hogwarts Server Terminal (Log Verification):

Displaying the `cat` output of the log file to verify all events were successfully written to disk.

```

→ HW6 git:(main) X cat server.log
[2026-05-20 21:35:04] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
[2026-05-20 21:35:06] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 21:35:06] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 21:35:21] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
[2026-05-20 21:35:27] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:35:27] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 21:35:32] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 21:35:32] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 21:35:39] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 21:35:39] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-20 21:36:27] [SERVER] TIMEOUT username=harry fd=5 elapsed=60s
[2026-05-20 21:36:27] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 21:36:32] [SERVER] TIMEOUT username=ron fd=6 elapsed=60s
[2026-05-20 21:36:32] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-20 21:36:39] [SERVER] TIMEOUT username=dumbledore fd=8 elapsed=60s
[2026-05-20 21:36:39] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-20 21:45:28] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 21:45:28] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 21:51:48] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
[2026-05-20 21:51:55] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:51:55] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 21:52:01] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 21:52:01] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 21:52:06] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 21:52:06] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-20 21:52:11] [SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 21:52:17] [SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 21:52:27] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-20 21:52:34] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 21:52:34] [SERVER] ENROLL username=neville type=WIZARD fd=5
[2026-05-20 21:52:42] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 21:52:58] [SERVER] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-20 21:52:59] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 21:53:00] [SERVER] CLIENT_DISCONNECTED username=neville reason=APPARATE
[2026-05-20 22:08:13] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 22:08:13] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 22:09:03] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-20 22:09:59] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:09:59] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 22:10:00] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:10:00] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-20 22:10:03] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 22:10:10] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-20 22:10:10] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 22:10:16] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-20 22:10:26] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-20 22:10:26] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-20 22:11:06] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 22:11:06] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 22:11:11] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-20 22:11:28] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:11:28] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 22:11:30] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:11:30] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-20 22:11:33] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 22:11:40] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-20 22:11:40] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 22:11:42] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-20 22:11:55] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:11:55] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 22:11:58] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-20 22:11:58] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-20 22:12:05] [SERVER] TIMEOUT username=ron fd=6 elapsed=10s
[2026-05-20 22:12:05] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-20 22:12:12] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:12:12] [SERVER] ENROLL username=ron type=WIZARD fd=5
[2026-05-20 22:12:14] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-20 22:12:19] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 22:12:19] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 22:12:20] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-20 22:12:24] [SERVER] TIMEOUT username=ron fd=5 elapsed=10s
[2026-05-20 22:12:24] [SERVER] CLIENT_DISCONNECTED username=ron reason=timeout
[2026-05-20 22:12:30] [SERVER] TIMEOUT username=dumbledore fd=6 elapsed=10s
[2026-05-20 22:12:30] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-20 22:12:31] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 22:12:31] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=5
[2026-05-20 22:12:33] [SERVER] ROSTER professor=mcgonagall clients=1
[2026-05-20 22:12:43] [SERVER] TIMEOUT username=mcgonagall fd=5 elapsed=10s
[2026-05-20 22:12:43] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
[2026-05-20 22:12:52] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 22:12:52] [SERVER] CLEANUP_DONE clients=0
→ HW6 git:(main) X |

```

Figure 14: Scenario 4 - Server Log File Output

3. Hermione (Wizard) Terminal:

Executing commands and eventually dropping due to inactivity.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
BREW MOONSTONE 5
[WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
BREW DRAGON_SCALE 2
[WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
[WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X |
```

Figure 15: Scenario 4 - Hermione Client Session

4. Harry (Wizard) Terminal:

Sitting entirely idle and being cleanly disconnected by the server's timeout engine.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
[WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X |
```

Figure 16: Scenario 4 - Harry Client Session (Disconnected via Timeout)

5. Ron (Wizard) Terminal:

Submitting a BREW command before timing out.

```
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
[WIZARD ron] RECEIVED TIMEOUT DISCONNECT
[WIZARD ron] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
BREW UNICORN_HAIR 5
[WIZARD ron] SENT BREW UNICORN_HAIR 5
[WIZARD ron] RECEIVED OK BREW UNICORN_HAIR 5 25
[WIZARD ron] RECEIVED TIMEOUT DISCONNECT
[WIZARD ron] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X |
```

Figure 17: Scenario 4 - Ron Client Session

6. Dumbledore (Professor) Terminal:

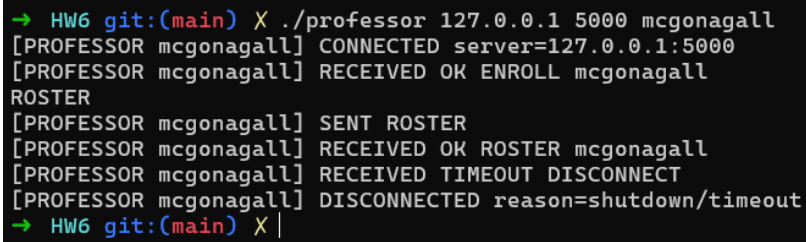
Executing SCROLL before the timeout threshold is reached.

```
→ HW6 git:(main) X ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
SCROLL
[PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
[PROFESSOR dumbledore] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR dumbledore] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X |
```

Figure 18: Scenario 4 - Dumbledore Client Session

7. McGonagall (Professor) Terminal:

Executing ROSTER before being pruned by the system.



```
→ HW6 git:(main) X ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER mcgonagall
[PROFESSOR mcgonagall] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR mcgonagall] DISCONNECTED reason=shutdown/timeout
→ HW6 git:(main) X |
```

Figure 19: Scenario 4 - McGonagall Client Session

6 Challenges and Engineering Solutions

6.1 Challenge 1: POSIX Structure Masking Under Strict C11 Standards

Root Problem Context: When configuring compilation metrics using the mandatory compiler flags `gcc -Wall -Wextra -std=c11`, the compiler threw structural errors, such as: `error: storage size of sa isn't known`. This occurred because compiling under strict ISO C11 standard configurations automatically masks non-core POSIX system extension definitions, hiding vital interfaces like `struct sigaction`, `sigemptyset()`, and `getopt()`.

Resolution Strategy: This issue was resolved by introducing an explicit feature test macro definition at the absolute beginning of all translation units (`.c` files), prior to importing any system headers:

```
1 #define _POSIX_C_SOURCE 200809L
```

This explicitly directs the compiler toolchain to expose the IEEE Std 1003.1-2008 system call extensions while fully preserving strict ISO C11 compliance. This resolved all compiler warnings and errors.

6.2 Challenge 2: Multi-Command Packet Aggregation and Stream Boundary Collisions

Root Problem Context: During testing with rapid command sequences, multiple messages occasionally merged into a single network chunk within the TCP layer. Initially, the server processed only the first command and reset the stream tracking indices. This caused the remaining buffered commands to be corrupted or lost, leading to protocol desynchronization and broken state flows.

Resolution Strategy: The parsing layer was refactored to replace simple index markers with an exhaustive continuous token checking engine. The system was modified to wrap the line parser in an infinite loop that scans for a newline character using `memchr()`. When found, the command is extracted and processed. The remaining unparsed block is then shifted to the front of the array using safe byte-block movement:

```
1 memmove(c->line_buf, c->line_buf + consumed, remain);
```

This completely isolates message processing from network packet boundaries, safely handling both fragmented lines and multi-statement chunks.